

Buổi 2 – Luyện tập nâng cao với Git

(Kèm lời giải chi tiết)

Hoạt động 1: Ôn tập cơ bản

👉 Mục tiêu: Hiểu sự khác biệt giữa Working Directory, Staging Area và Repository.

****Lời giải:****

1. Tạo nhánh mới:

```
git checkout -b practice-branch
```

2. Tạo file:

```
echo "File cho staging + commit" > practice1.txt
```

```
echo "File cho staging, chưa commit" > practice2.txt
```

```
echo "File ở working directory" > practice3.txt
```

3. Thao tác:

```
git add practice1.txt
```

```
git commit -m "Add practice1"
```

```
git add practice2.txt # chưa commit
```

4. Quan sát trạng thái:

```
git status
```

```
git diff
```

```
git diff --staged
```

5. Khôi phục:

```
git restore practice3.txt
```

```
git restore --staged practice2.txt
```

6. Quản lý file:

```
git mv practice1.txt exercise1.txt
```

```
git commit -m "Rename practice1 to exercise1"
```

```
git rm exercise1.txt
```

```
git commit -m "Remove exercise1"
```

7. Push nhánh và tạo PR:

```
git push origin practice-branch
```

Hoạt động 2: Git Stash

👉 Mục tiêu: Lưu thay đổi tạm thời khi chưa muốn commit.

****Lời giải:****

1. Chỉnh sửa README.md.

```
echo "Thêm ghi chú buổi 2" >> README.md
```

2. Lưu stash:

```
git stash save "update readme"  
git stash list
```

3. Tạo commit khác:

```
echo "Ghi chú" > note.md  
git add note.md  
git commit -m "Add note.md"  
git push
```

4. Khôi phục:

```
git stash pop  
git add README.md  
git commit -m "Restore readme changes"  
git push
```

5. Stash nhiều lần:

```
echo "Stash 1" >> README.md  
git stash  
echo "Stash 2" >> README.md  
git stash  
git stash list
```

6. Quản lý stash:

```
git stash apply stash@{1}  
git stash drop stash@{1}  
git stash clear
```

Hoạt động 3: Revert & Reset

👉 Mục tiêu: Hiểu cách quay lại trạng thái trước đó bằng nhiều cách.

****Lời giải:****

1. Tạo file và commit nhiều lần:

```
echo "Dòng 1" > history.txt
```

```
git add . && git commit -m "Add line 1"
```

```
echo "Dòng 2" >> history.txt && git commit -am "Add line 2"
```

```
echo "Dòng 3" >> history.txt && git commit -am "Add line 3"
```

2. git log --oneline

3. git revert HEAD (tạo commit mới xóa dòng 3)

4. Thêm 2 commit nữa (dòng 4, dòng 5)

5. Reset thử các kiểu:

```
git reset --soft <commit-id>
```

```
git reset --mixed <commit-id>
```

```
git reset --hard <commit-id>
```

6. git reflog

7. Khôi phục commit đã mất bằng git checkout hoặc git reset.

Hoạt động 4: Tag và Release

👉 Mục tiêu: Biết cách đánh dấu mốc quan trọng của dự án.

****Lời giải:****

1. echo "Version 1.0" > version.txt

```
git add . && git commit -m "Add version 1.0"
```

```
git tag v1.0
```

```
git push origin v1.0
```

2. echo "Version 2.0" > version.txt

```
git commit -am "Update version 2.0"
```

```
git tag -a v2.0 -m "Release version 2.0"
```

```
git push origin v2.0
```

3. git tag -l

4. git tag beta-1.0

5. git tag -d beta-1.0

6. git push origin --delete v1.0

7. git checkout -b branch-from-v2 v2.0

Hoạt động 5: Rebase thay vì merge

👉 Mục tiêu: Làm lịch sử commit gọn gàng hơn.

****Lời giải:****

1. git checkout -b feature-branch
echo "Nội dung tính năng" > feature.md
git add . && git commit -m "Add feature.md"
git push origin feature-branch
2. git checkout main
echo "Main file" > main.md
git add . && git commit -m "Add main.md"
3. Merge:
git merge feature-branch
git log --graph --oneline --all
4. Reset lại main:
git reset --hard HEAD~1
5. Rebase:
git rebase feature-branch
git log --graph --oneline --all
6. Tạo conflict:
Sửa cùng dòng trong cả main.md và feature.md
git add . && git commit -m "Conflict commit"
git rebase feature-branch
7. Giải quyết conflict:
git add <file>
git rebase --continue
8. So sánh merge và rebase: Merge giữ lịch sử, Rebase làm lịch sử gọn hơn.

Yêu cầu nộp sau Buổi 2

- File Word ghi lại lệnh đã chạy, screenshot kết quả, và giải thích.
- Link GitHub repo sau buổi 2 (có commit, nhánh, tag, rebase).